Tcpdump — Study Guide

Tcpdump is the classic command-line **packet capture and analysis tool** — the foundational skill underneath every "network traffic analysis" task you'll do as a SOC analyst, and the CLI counterpart to Wireshark (which you'll likely study next, since they use the same underlying capture engine — libpcap). Where everything in your last series was about finding and prioritizing vulnerabilities, tcpdump is about actually *seeing* what's happening on the wire — essential for both offensive verification (confirming an exploit's traffic) and defensive investigation (SOC packet analysis).

---

1. What tcpdump Actually Does

tcpdump captures packets flowing across a network interface and prints them to your terminal (or saves them to a file) in a human-readable or raw format. It uses **libpcap** under the hood — the same packet-capture library Wireshark, Nmap, and countless other tools rely on, which is why capture files are cross-compatible between them.

```
tcpdump   → CLI, lightweight, great for live/remote/scripted capture
Wireshark → GUI, deeper protocol dissection, better for deep manual analysis
```

A very common real-world workflow: capture with tcpdump on a remote server (no GUI available), then transfer the `.pcap` file to your workstation and open it in Wireshark for deep analysis.

---

2. Basic Usage

```
tcpdump                          # capture on default inter
face, print to terminal
sudo tcpdump -i eth0              # capture on a specific i
nterface
tcpdump -D                        # list all available int
erfaces
```

Root/sudo privileges are required for live capture on most systems, since reading raw packets off an interface is a privileged operation.

3. Key Flags

Flag	Meaning
`-i`	Interface to capture on (`-i any` captures all interfaces)
`-n`	Don't resolve hostnames (faster, avoids DNS lookups triggering their own traffic)
`-nn`	Don't resolve hostnames OR port names (shows raw port numbers, e.g. `80` not `http`)
`-v` / `-vv` / `-vvv`	Increasing verbosity
`-c`	Capture only N packets, then stop
`-w`	Write raw capture to a file (`.pcap`)
`-r`	Read from a previously saved `.pcap` file
`-A`	Print packet contents in ASCII (useful for reading plaintext protocols)
`-X`	Print packet contents in hex AND ASCII
`-s`	Snap length — how many bytes to capture per packet (`-s0` or `-s 262144` = capture full packet)
`-e`	Print link-layer (Ethernet) header too
`-q`	Quiet — less protocol detail per line
`-tttt`	Print full, readable timestamps

4. Practical First Commands

```
sudo tcpdump -i eth0 -n -c 20                # quick 20-pack
et look at traffic
sudo tcpdump -i eth0 -n -w capture.pcap       # save to fil
e for later analysis
tcpdump -r capture.pcap                       # read it bac
k
sudo tcpdump -i eth0 -nn -A                  # readable tex
t, no name resolution
```

5. Berkeley Packet Filter (BPF) Syntax — Filtering Traffic

Capturing everything is noisy and unmanageable — BPF filter expressions let you narrow captures to exactly what you care about. This is the single most important skill in tcpdump.

a) Host and Network Filters

```
tcpdump host 192.168.1.10                    # traffic to/from
a specific host
tcpdump src 192.168.1.10                      # traffic FROM th
is host only
tcpdump dst 192.168.1.10                       # traffic TO thi
s host only
tcpdump net 192.168.1.0/24                      # traffic withi
n a subnet
```

b) Port Filters

```
tcpdump port 80                              # traffic on port
80 (either direction)
tcpdump src port 443                           # traffic source
d from port 443
tcpdump portrange 1-1024                        # port range
```

c) Protocol Filters

```
tcpdump tcp
tcpdump udp
tcpdump icmp
tcpdump arp
```

d) Combining Filters with Logical Operators

```
tcpdump host 192.168.1.10 and port 80
tcpdump src 192.168.1.10 and dst port 443
```

```
tcpdump tcp and not port 22                    # everything T
CP except SSH
tcpdump 'host 192.168.1.10 or host 192.168.1.20'
```

e) Flag-Based Filters (TCP flags — genuinely high-value for security analysis)

```
tcpdump 'tcp[tcpflags] == tcp-syn'          # only SYN packe
ts (connection attempts)
tcpdump 'tcp[tcpflags] & tcp-syn != 0'              # SYN
flag set (incl. SYN-ACK)
tcpdump 'tcp[tcpflags] == tcp-rst'          # only RST packe
ts (connection resets)
```

Why this matters for detection: a flood of SYN packets with no completing handshake is the classic signature of a SYN flood DoS attack or an aggressive port scan (like Nmap's default `-sS` SYN scan) — this filter is exactly how you'd spot that pattern in a capture.

---

6. Practical Recon/Security Use Cases

a) Watching an Nmap Scan Land (Great Learning Exercise)

Run this on your Metasploitable 2 VM (or any lab target) while scanning it from your Kali box:

```
sudo tcpdump -i eth0 -n 'tcp[tcpflags] == tcp-syn'
```

Then run `nmap -sS target` from another machine — you'll watch the SYN packets from the scan arrive in real time, seeing exactly what a port scan looks like from the *target's* perspective. This is a genuinely good way to internalize what your earlier Nmap guide's scan types actually do at the packet level.

b) Capturing Plaintext Credentials (Demonstrating Why Encryption Matters)

```
sudo tcpdump -i eth0 -A port 21            # watch FTP traff
ic in plaintext
```

```
sudo tcpdump -i eth0 -A port 23          # watch Telnet tr
affic in plaintext
```

On a lab environment where you control both ends, logging into an FTP/Telnet service while this capture runs will show the **username and password in cleartext** directly in the terminal output — one of the most effective ways to *demonstrate* (not just explain) why plaintext protocols are a finding worth flagging in a VAPT report.

c) DNS Query Monitoring

```
sudo tcpdump -i eth0 -n port 53
```

Useful for spotting unusual DNS activity — a host resolving hundreds of random-looking domains in a short window is a classic DGA (Domain Generation Algorithm) malware/C2 indicator, exactly the kind of pattern your DNS enumeration guide's defensive section referenced.

d) Watching a Reverse Shell Connection

```
sudo tcpdump -i eth0 -n port 4444
```

If you're practicing the Netcat reverse shell workflow from your earlier guide, running this alongside lets you *see* the connection establish at the packet level — reinforcing the Kill Chain's "Command & Control" stage as something concrete and observable, not just a theoretical concept.

---

7. Writing and Reading Capture Files

```
# Write a capture to disk (best practice: use -w for anythi
ng you'll analyze later)
sudo tcpdump -i eth0 -w suspicious_traffic.pcap

# Read it back with filters applied on the saved file
tcpdump -r suspicious_traffic.pcap -n port 445

# Rotate capture files (useful for long-running captures -
avoids one giant file)
```

```
sudo tcpdump -i eth0 -w capture -C 100        # new file ev
ery 100MB
sudo tcpdump -i eth0 -w capture -G 3600 -w 'capture-%Y%m%d-
%H%M%S.pcap' # new file every hour
```

Why `-w` matters so much in practice: raw `.pcap` files preserve every byte of every packet, letting you apply *different* filters after the fact, or open the same capture in Wireshark for deeper protocol dissection — printing directly to terminal (without `-w`) loses this flexibility entirely.

---

8. Reading tcpdump Output — Understanding a Line

```
14:32:07.123456 IP 192.168.1.50.54321 > 192.168.1.10.80: Fl
ags [S], seq 123456789, win 64240, length 0
```

Breaking this down:

```
14:32:07.123456   → timestamp
IP                 → protocol (IPv4)
192.168.1.50.54321 → source IP.port
> 192.168.1.10.80  → destination IP.port
Flags [S]           → TCP flag(s) - S=SYN, A=ACK, P=PSH, F=
FIN, R=RST
seq 123456789         → sequence number
win 64240              → TCP window size
length 0                → payload length (0 = no data, just
a control packet)
```

TCP flag reference (worth memorizing):

```
S  = SYN   (connection initiation)
A  = ACK   (acknowledgment)
P  = PSH   (push data immediately)
F  = FIN   (graceful connection close)
R  = RST   (abrupt connection reset)
.  = (no flags shown, typically just ACK on established con
nections)
```

Recognizing a normal TCP handshake pattern (`S` → `S.` (SYN-ACK) → `.` (ACK)) versus something abnormal (lots of unanswered `S` packets, or unexpected `R` resets) is a core packet-analysis skill.

---

9. tcpdump vs. Wireshark — Where Each Fits

	tcpdump	Wireshark
Interface	CLI only	GUI (with a CLI counterpart, `tshark`)
Best for	Remote servers, scripting, quick live checks, resource-constrained environments	Deep manual protocol analysis, following TCP streams, visual inspection
Learning curve	BPF syntax takes practice	Filter syntax is different (Wireshark display filters ≠ BPF capture filters) but GUI makes exploration easier
Typical workflow	Capture (`-w`) on the target/server	Analyze the resulting `.pcap` afterward

This is exactly the "capture on the server, analyze on the workstation" pattern mentioned in Section 1 — worth internalizing as the standard operating pattern, since you often won't have GUI access on the systems where you actually need to capture traffic (production servers, cloud instances, remote boxes).

---

10. tcpdump for Defensive/SOC Work

Since you're building SOC skills specifically, a few angles worth knowing:

a) Baseline traffic understanding — before you can spot anomalous traffic, you need to know what "normal" looks like on a given network; running periodic captures and reviewing them builds this intuition over time.

b) Incident response packet capture — during an active investigation, tcpdump is often the fastest way to start capturing evidence on a potentially compromised host while more thorough tools (Wireshark, full EDR telemetry) are being set up.

c) Confirming IDS/IPS alerts — a SIEM/IDS alert says "possible port scan from X" — a quick `tcpdump host X` capture can confirm/deny and provide the raw evidence for a ticket.

d) Detecting exfiltration patterns — unusually large outbound data transfers to an unfamiliar destination, especially over unusual ports, is exactly the kind of

thing a targeted tcpdump capture (`tcpdump -n dst host <suspicious_ip>`) helps confirm.

11. Practical Example — Full Investigative Workflow

```
# 1. Something looks suspicious - start a capture immediate
ly, don't wait
sudo tcpdump -i eth0 -w incident_$(date +%Y%m%d_%H%M%S).pca
p

# 2. In a separate terminal, narrow in on the suspicious ho
st while capture runs
sudo tcpdump -i eth0 -n host 192.168.1.50 -c 50

# 3. Once you have enough, stop the full capture (Ctrl+C) a
nd dig into the saved file
tcpdump -r incident_20260717_143207.pcap -nn 'tcp[tcpflags]
== tcp-syn' | wc -l   # count SYN packets - scan indicator?

# 4. Pull out plaintext protocol traffic for review
tcpdump -r incident_20260717_143207.pcap -A port 21

# 5. Hand off the .pcap to Wireshark for deeper stream reco
nstruction if needed
```

12. Kill Chain / ATT&CK Mapping

Context	Framework Mapping
Using tcpdump to scan/verify traffic (offensive)	Supports Reconnaissance verification, C2 confirmation
Using tcpdump defensively (SOC)	Maps to detection/analysis activities across **T1046** (Network Service Discovery being observed), **T1071** (C2 traffic being captured), and general network traffic analysis defenses referenced across most ATT&CK technique mitigation sections

Quick Reference Cheat Sheet

```
tcpdump -D                                    # list interfac
es
sudo tcpdump -i eth0                           # capture on i
nterface
sudo tcpdump -i eth0 -n -c 20                  # 20 packets,
no DNS resolution
sudo tcpdump -i eth0 -w capture.pcap           # save to fi
le
tcpdump -r capture.pcap                        # read from
file
sudo tcpdump -i eth0 -A port 21                # plaintex
t protocol viewing
sudo tcpdump -i eth0 -nn 'tcp[tcpflags] == tcp-syn'   # SYN
packets only

# Filters
host X / src X / dst X
port X / src port X / portrange 1-1024
tcp / udp / icmp / arp
and / or / not
```